

A Type System for STPA

Well-formedness and coverage as typing judgments

Claude Opus 4.8 (Anthropic)

June 24, 2026

Abstract

We give a lightweight type system for documents produced by System-Theoretic Process Analysis (STPA). The artifacts of an STPA analysis form a fixed dependency chain (losses, hazards, constraints, a control structure, unsafe control actions, controller requirements, loss scenarios). We separate the *authority relation* over the control structure, which must be acyclic, from the *information-flow relation*, which may contain cycles. Typing rules then enforce referential well-kindedness and *totality*: every artifact must be linked to its required parent, and—the central concern— every Unsafe Control Action (UCA) must propagate down to at least one controller constraint or requirement. An orphaned UCA is precisely a non-exhaustiveness error.

1 Syntax and kinds

A document $\mathcal{D}$ is a finite set of typed nodes together with a set of directed edges. We distinguish entity kinds

$K ::= \text{Loss} \mid \text{Hazard} \mid \text{Constraint} \mid \text{Controller} \mid \text{ControlAction} \mid \text{Feedback} \mid \text{UCA} \mid \text{Requirement} \mid \text{Scenario}.$

A UCA is additionally indexed by one of the four STPA guide phrasings:

$g ::= \text{NotProvided} \mid \text{Provided} \mid \text{WrongTiming} \mid \text{WrongDuration}.$

Two relations on the same node set. We keep the physical/information topology and the authority topology apart, because they support different judgments.

- $c \xrightarrow{\text{ctrl}} a$: controller c issues control action a (authority). The transitive closure of $\xrightarrow{\text{ctrl}}$ on controllers is required to be *acyclic* — this is the DAG.
- $s \xrightarrow{\text{fb}} c$: sensor/feedback s informs controller c . Together with $\xrightarrow{\text{ctrl}}$ this closes the loop, so $\xrightarrow{\text{fb}} \cup \xrightarrow{\text{ctrl}}$ may contain *cycles*. We never check reachability over this union; we check coverage.
- $x \sqsubseteq y$: artifact x refines / derives artifact y (e.g. a requirement refines a UCA).
- $x \rightsquigarrow y$: x traces back to y (e.g. a UCA traces to a hazard).

The type of a UCA. The judgment-bearing object is the UCA, whose type records its issuing controller, its control action, its guide word, and a non-empty set of hazards it traces to:

$\text{UCA}(a : \text{ControlAction}, g, \text{by} : \text{Controller}, \text{traces} : \text{NonEmpty}[\text{Hazard}]).$

2 Judgment forms

$\mathcal{D} \vdash \mathcal{D} \text{ ok}$ the document is well-formed
 $\mathcal{D} \vdash n : K$ node n has kind K in $\mathcal{D}$
 $\mathcal{D} \vdash x \sqsubseteq y$ x derives y
 $\mathcal{D} \vdash x \rightsquigarrow y$ x traces back to y

3 Backward-traceability rules (totality upward)

Every artifact below the top of the chain must cite a parent. These are the “every child has a parent” obligations.

$$\text{HAZ-LOSS} \frac{\mathcal{D} \vdash h : \text{Hazard} \quad \mathcal{D} \vdash \ell : \text{Loss} \quad \mathcal{D} \vdash h \rightsquigarrow \ell}{\mathcal{D} \vdash h \text{ ok}}$$

$$\text{CON-HAZ} \frac{\mathcal{D} \vdash k : \text{Constraint} \quad \mathcal{D} \vdash h : \text{Hazard} \quad \mathcal{D} \vdash k \sqsubseteq h}{\mathcal{D} \vdash k \text{ ok}}$$

$$\text{UCA-WF} \frac{\mathcal{D} \vdash u : \text{UCA} \quad \mathcal{D} \vdash a : \text{ControlAction} \quad \mathcal{D} \vdash c : \text{Controller} \quad \mathcal{D} \vdash h : \text{Hazard} \quad u = \text{UCA}(a, g, \text{by}=c, \text{traces} \ni h) \quad \mathcal{D} \vdash c \xrightarrow{\text{ctrl}} a \quad \mathcal{D} \vdash u \rightsquigarrow h}{\mathcal{D} \vdash u \text{ ok}}$$

UCA-WF is the workhorse. It simultaneously enforces (i) the UCA names a controller $\text{by}=c$ (the structured-editor invariant), (ii) referential well-kindedness: the named controller must actually issue the cited action, $c \xrightarrow{\text{ctrl}} a$ (a join against the control-structure DAG), and (iii) at least one hazard in scope, since traces is `NonEmpty`.

4 Forward-propagation rules (totality downward)

This is the central concern: a UCA must reach a controller constraint or a requirement. We introduce a *derivation* judgment and then the coverage obligation.

$$\text{REQ-DERIVES} \frac{\mathcal{D} \vdash r : \text{Requirement} \quad \mathcal{D} \vdash u : \text{UCA} \quad \mathcal{D} \vdash r \sqsubseteq u}{\mathcal{D} \vdash u \triangleright r}$$

$$\text{CON-DERIVES} \frac{\mathcal{D} \vdash k : \text{Constraint} \quad \mathcal{D} \vdash u : \text{UCA} \quad \mathcal{D} \vdash k \sqsubseteq u}{\mathcal{D} \vdash u \triangleright k}$$

where $u \triangleright x$ reads “the UCA u propagates to mitigation x .” The coverage rule says a well-formed UCA must have *some* such x :

$$\text{UCA-COVERED} \frac{\mathcal{D} \vdash u \text{ ok} \quad \exists x. \mathcal{D} \vdash u \triangleright x}{\mathcal{D} \vdash \text{covered}(u)}$$

The negation is the diagnostic you want raised by the checker:

$$\text{ORPHAN-UCA (ERROR/LINT)} \frac{\mathcal{D} \vdash u \text{ ok} \quad \nexists x. \mathcal{D} \vdash u \triangleright x}{\mathcal{D} \vdash \text{orphan}(u)}$$

$\text{orphan}(u)$ is exactly a non-exhaustive-match error: a case (the UCA) with no handler (no derived requirement). Dually, a requirement that derives no UCA is dead code:

$$\text{DEAD-REQ (LINT)} \frac{\mathcal{D} \vdash r : \text{Requirement} \quad \nexists u. \mathcal{D} \vdash r \sqsubseteq u}{\mathcal{D} \vdash \text{dead}(r)}$$

5 Loss scenarios

A loss scenario must explain how a UCA occurs, and its causal factors should attach to edges of the control structure.

$$\text{SCN-WF} \frac{\mathcal{D} \vdash s : \text{Scenario} \quad \mathcal{D} \vdash u : \text{UCA} \quad \mathcal{D} \vdash s \rightsquigarrow u \quad \mathcal{D} \vdash u \text{ ok}}{\mathcal{D} \vdash s \text{ ok}}$$

6 Acyclicity of the authority relation

Let $\xrightarrow{\text{ctrl}}^+$ be the transitive closure of $\xrightarrow{\text{ctrl}}$ restricted to controllers. Acyclicity is a side condition on document well-formedness, not a per-node rule.

$$\text{AUTHORITY-DAG} \frac{\forall c. \mathcal{D} \vdash \neg (c \xrightarrow{\text{ctrl}}^+ c)}{\mathcal{D} \vdash \text{acyclic}(\xrightarrow{\text{ctrl}})}$$

Note the absence of any analogous rule for $\xrightarrow{\text{fb}} \cup \xrightarrow{\text{ctrl}}$: information flow is allowed to cycle, so no acyclicity obligation is imposed on it.

7 Document well-formedness

The whole document typechecks when every node is well-formed, every UCA is covered, and the authority relation is acyclic.

$$\text{DOC-WF} \frac{\begin{array}{c} (\forall n \in \mathcal{D}. \mathcal{D} \vdash n \text{ ok}) \\ (\forall u. \mathcal{D} \vdash u : \text{UCA} \implies \mathcal{D} \vdash \text{covered}(u)) \\ \mathcal{D} \vdash \text{acyclic}(\xrightarrow{\text{ctrl}}) \end{array}}{\mathcal{D} \vdash \mathcal{D} \text{ ok}}$$

8 Authoring gate vs. review gate

Remark 1. *During authoring a partially specified UCA is normal, so ORPHAN-UCA should fire as a non-blocking lint (cf. an unused-variable warning). At a “ready-for-review” gate it is promoted to an error that blocks DOC-WF. The two gates share the same rules; only the severity of orphan(·) and dead(·) changes.*

9 Operational reading: a Datalog encoding

The rules above translate almost verbatim into Datalog, where the coverage and orphan checks are recursive queries rather than hand-written traversals.

```
% facts extracted from the document
uca(U). control_action(A). controller(C). hazard(H).
issues(C, A).           % c --ctrl--> a
uca_by(U, C).           % UCA cites controller C
uca_action(U, A).       % UCA is about action A
uca_traces(U, H).       % UCA traces to hazard H
refines(X, Y).          % requirement/constraint X refines Y

% UCA-Wf: referential well-kindedness
uca_wf(U) :- uca(U), uca_by(U, C), uca_action(U, A), issues(C, A),
             uca_traces(U, _).

% propagation
derives(U, X) :- refines(X, U), uca(U).

% the headline checks
covered(U)    :- uca_wf(U), derives(U, _).
orphan_uca(U) :- uca_wf(U), not derives(U, _).
dead_req(R)   :- requirement(R), not refines(R, _).

% authority acyclicity (cycle => report)
reaches(C1, C2) :- issues_ctrl(C1, C2).
reaches(C1, C3) :- reaches(C1, C2), issues_ctrl(C2, C3).
authority_cycle(C) :- reaches(C, C).
```

Equivalently, in a typed-DSL setting the same guarantees are obtained by *smart constructors*: a UCA value cannot be built without a controller and a non-empty hazard set, which makes UCA-WF’s referential conditions unrepresentable as failures rather than detectable after the fact. The structured editor and the typechecker are then the same invariant enforced at two times—construction time and whole-document time.